Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 – «Информатика, вычислительная техника»

Дискретная математика и математическая логика
Лабораторная работа №2
«Алгоритм Прима»

Выполнил студент гр. ИВТ-24-1б:

Оглезнев Никита Михайлович
(Фамилия, Имя, Отчество)

(подпись)

Проверил:

ст. преп. каф. ИТАС
Рустамханова Г. И.
(должность, Ф.И.О руководителя от кафедры)

(оценка) (подпись)

(дата)

Пермь 2026

Содержание

Содержание 2
Цель 3
Задачи 3
Теоретический блок 4
Реализация 5
Заключение 8
Приложение А 9
Приложение Б 11

Цель

Реализация алгоритма Прима для поиска минимального остовного дерева

Задачи

1. Реализовать алгоритм Прима для обработки матрицы смежности размером 10x10
2. Организовать корректный ввод данных из внешнего текстового файла или использование тестового набора данных.
3. Обеспечить вывод структуры полученного остовного дерева (список ребер) и его суммарного веса.
4. Предусмотреть обработку исключений (отсутствие файла, некорректный формат данных, несвязность графа).

Теоретический блок

1. Минимальное остовное дерево

Минимальное остовное дерево (MST) — это подмножество ребер связного взвешенного неориентированного графа, которое соединяет все его вершины без образования циклов и имеет минимально возможный суммарный вес ребер.

2. Алгоритм Прима

Алгоритм Прима — итерационный «жадный» алгоритм. Основные этапы:

1. Выбирается произвольная начальная вершина, которая включается в дерево.
2. На каждом шаге рассматриваются все ребра, соединяющие вершины из дерева с вершинами, еще не включенными в него.
3. Из этих ребер выбирается ребро с минимальным весом, и соответствующая вершина добавляется в дерево.
4. Процесс повторяется до тех пор, пока все вершины не будут включены в дерево (всего будет выбрано n-1 ребро).

Реализация

Программа построена как последовательная обработка графа, начиная с этапа сбора и валидации входных данных.

На первом этапе программа запрашивает у пользователя источник информации: это может быть текстовый файл, где веса ребер представлены в виде квадратной таблицы, или заранее подготовленный тестовый набор данных. Функция чтения файла строго контролирует структуру матрицы, проверяя её размерность 10x10, отсутствие отрицательных весов и симметричность значений, что критически важно для корректного описания неориентированного графа. Если данные проходят проверку, они передаются в функцию вывода, которая форматирует матрицу смежности в виде аккуратной таблицы для визуального контроля исходных условий.

Центральным узлом программы является реализация алгоритма Прима (рис.1), который работает по принципу постепенного расширения «области влияния» остовного дерева. В самом начале создается логический массив посещенных вершин, где изначально отмечена только стартовая точка. Основной цикл программы итеративно ищет самое легкое ребро, один конец которого уже находится внутри растущего дерева, а другой ведет к еще не охваченной вершине. Это достигается за счет вложенных циклов, которые перебирают все возможные связи между текущим подмножеством остова и остальными вершинами графа. Как только такое минимальное ребро найдено, целевая вершина помечается как посещенная, а само ребро фиксируется в результирующем списке.

```python
def prim(matrix, start=0):
    """
    Алгоритм Прима для нахождения минимального остовного дерева.
    Возвращает (список рёбер, общий вес).
    Каждое ребро представляется кортежем (вершина1, вершина2, вес).
    Вершины нумеруются от 0 до 9.
    """
    n = len(matrix)

    # visited[v] == True означает, что вершина v уже включена в остов.
    visited = [False] * n

    # Начинаем построение дерева с вершины start.
    visited[start] = True

    # Здесь будем накапливать выбранные рёбра остова.
    mst_edges = []

    # Общий вес минимального остовного дерева.
    total_weight = 0

    # В связном графе остов для n вершин всегда содержит n - 1 ребро.
    for _ in range(n - 1):
        # На каждой итерации ищем самое дешёвое ребро,
        # которое соединяет уже посещённую вершину с непосещённой.
        min_weight = float('inf')
        from_vertex = -1
        to_vertex = -1

        # Ищем минимальное ребро из уже включённых вершин
        # в вершину, которая ещё не входит в остов.
        for u in range(n):
            # Рассматриваем только вершины, которые уже попали в дерево.
            if not visited[u]:
                continue
            for v in range(n):
                w = matrix[u][v]

                # Пропускаем:
                # 1) вершины, которые уже находятся в остове;
                # 2) отсутствующие рёбра (в матрице это 0).
                if visited[v] or w == 0:
                    continue

                # Если нашли ребро легче текущего кандидата,
                # запоминаем его как лучшее на этой итерации.
                if w < min_weight:
                    min_weight = w
                    from_vertex = u
                    to_vertex = v

        # Если подходящее ребро не найдено, значит из текущей компоненты
        # нельзя дойти до остальных вершин: граф несвязный.
        if to_vertex == -1:
            print("Внимание: граф несвязный, построено только частичное дерево.")
            break

        # Добавляем найденное минимальное ребро в остов,
        # помечаем новую вершину как посещённую
        # и увеличиваем суммарный вес дерева.
        visited[to_vertex] = True
        mst_edges.append((from_vertex, to_vertex, min_weight))
        total_weight += min_weight

    return mst_edges, total_weight
```

Рисунок 1 – Программная реализация алгоритма Прима

В процессе работы алгоритм также аккуратно суммирует веса всех добавленных участков, формируя итоговую стоимость MST. Если на очередном шаге программа не может найти ни одного ребра, ведущего к новым вершинам, она выдает предупреждение о несвязности графа, что предотвращает бесконечное зацикливание. Финальный этап работы заключается в преобразовании внутренних индексов (от 0 до 9) в человекочитаемый формат (от 1 до 10) и выводе подробного отчета о каждом включенном ребре. Таким образом, программа представляет собой завершенный инструмент, который превращает абстрактную весовую матрицу в конкретную минимальную структуру соединений, минимизирующую общие затраты на построение сети.

Заключение

В ходе выполнения лабораторной работы была изучена и реализована программная модель алгоритма Прима, предназначенная для построения минимального остовного дерева во взвешенном неориентированном графе. Реализованный код успешно решает задачу оптимизации связей, позволяя объединить все узлы графа с минимальными суммарными затратами, что имеет прямое практическое применение в проектировании коммуникационных и дорожных сетей.

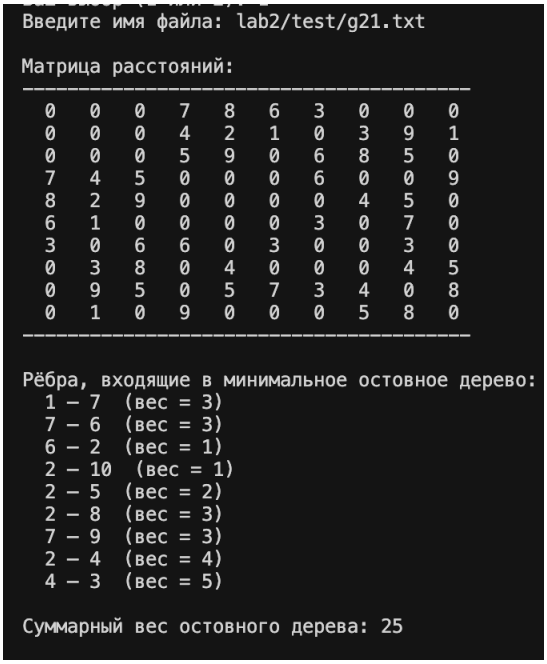
Программа продемонстрировала высокую надежность при обработке различных сценариев: она корректно считывает данные из внешних файлов, проверяет их на логическую целостность и симметричность, а также визуализирует процесс подбора ребер. Особое внимание было уделено обработке исключительных ситуаций, таких как несвязность графа или наличие отрицательных весов, что делает программный продукт устойчивым к ошибкам пользователя.

Результаты тестирования на матрице смежности 10x10 подтвердили теоретические положения алгоритма: итоговое дерево всегда содержит n-1 ребро и не имеет циклов, обеспечивая при этом глобальный минимум веса для данной конфигурации графа.

Таким образом, цель работы достигнута, а полученные навыки разработки и отладки графовых алгоритмов могут быть использованы при решении более сложных задач комбинаторной оптимизации.

Приложение А

Результаты работы программы на тестовых файлах.

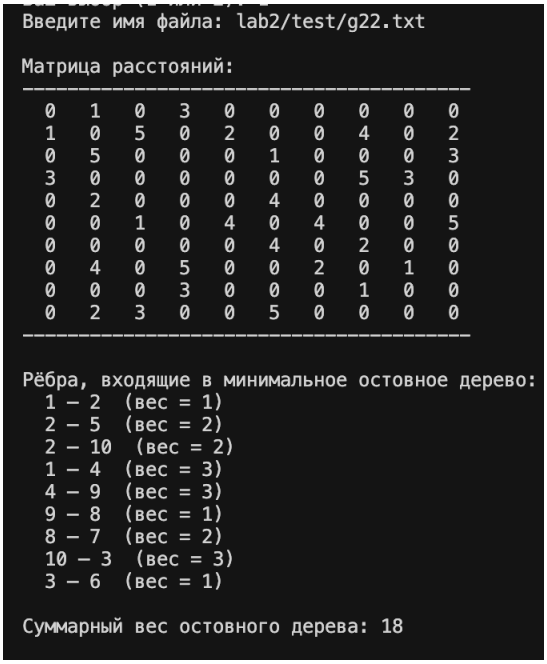


```
Введите имя файла: lab2/test/g21.txt

Матрица расстояний:
-----------------------------------------
  0   0   0   7   8   6   3   0   0   0
  0   0   0   4   2   1   0   3   9   1
  0   0   0   5   9   0   6   8   5   0
  7   4   5   0   0   0   6   0   0   9
  8   2   9   0   0   0   0   4   5   0
  6   1   0   0   0   0   3   0   7   0
  3   0   6   6   0   3   0   0   3   0
  0   3   8   0   4   0   0   0   4   5
  0   9   5   0   5   7   3   4   0   8
  0   1   0   9   0   0   0   5   8   0
-----------------------------------------

Рёбра, входящие в минимальное остовное дерево:
  1 — 7  (вес = 3)
  7 — 6  (вес = 3)
  6 — 2  (вес = 1)
  2 — 10  (вес = 1)
  2 — 5  (вес = 2)
  2 — 8  (вес = 3)
  7 — 9  (вес = 3)
  2 — 4  (вес = 4)
  4 — 3  (вес = 5)

Суммарный вес остовного дерева: 25
```



```
Введите имя файла: lab2/test/g22.txt

Матрица расстояний:
-----------------------------------------
  0   1   0   3   0   0   0   0   0   0
  1   0   5   0   2   0   0   4   0   2
  0   5   0   0   0   1   0   0   0   3
  3   0   0   0   0   0   0   5   3   0
  0   2   0   0   0   4   0   0   0   0
  0   0   1   0   4   0   4   0   0   5
  0   0   0   0   0   4   0   2   0   0
  0   4   0   5   0   0   2   0   1   0
  0   0   0   3   0   0   0   1   0   0
  0   2   3   0   0   5   0   0   0   0
-----------------------------------------

Рёбра, входящие в минимальное остовное дерево:
  1 — 2  (вес = 1)
  2 — 5  (вес = 2)
  2 — 10  (вес = 2)
  1 — 4  (вес = 3)
  4 — 9  (вес = 3)
  9 — 8  (вес = 1)
  8 — 7  (вес = 2)
  10 — 3  (вес = 3)
  3 — 6  (вес = 1)

Суммарный вес остовного дерева: 18
```

```
Введите имя файла: lab2/test/g23.txt

Матрица расстояний:
------------------------------------------
  0   3   4   2   0   0   0   0   0   0
  3   0   0   0   3   0   0   0   0   0
  4   0   0   0   6   0   0   0   0   0
  2   0   0   0   3   1   0   0   0   0
  0   3   6   3   0   1   8   7   0   0
  0   0   0   1   1   0   6   0   2   0
  0   0   0   0   8   6   0   0   0   4
  0   0   0   0   7   0   0   0   6   3
  0   0   0   0   0   2   0   6   0   5
  0   0   0   0   0   0   4   3   5   0
------------------------------------------

Рёбра, входящие в минимальное остовное дерево:
  1 — 4  (вес = 2)
  4 — 6  (вес = 1)
  6 — 5  (вес = 1)
  6 — 9  (вес = 2)
  1 — 2  (вес = 3)
  1 — 3  (вес = 4)
  9 — 10  (вес = 5)
  10 — 8  (вес = 3)
  10 — 7  (вес = 4)

Суммарный вес остовного дерева: 25
```

```
Введите имя файла: lab2/test/g24.txt

Матрица расстояний:
------------------------------------------
  0   2   3   4   0   0   0   0   0   0
  2   0   0   0   2   5   0   0   0   0
  3   0   0   0   3   0   0   0   0   0
  4   0   0   0   6   0   0   0   0   0
  0   2   3   6   0   1   8   7   0   0
  0   5   0   0   1   0   6   0  12   0
  0   0   0   0   8   6   0   0   0   4
  0   0   0   0   7   0   0   0   6   3
  0   0   0   0   0  12   0   6   0  11
  0   0   0   0   0   0   4   3  11   0
------------------------------------------

Рёбра, входящие в минимальное остовное дерево:
  1 — 2  (вес = 2)
  2 — 5  (вес = 2)
  5 — 6  (вес = 1)
  1 — 3  (вес = 3)
  1 — 4  (вес = 4)
  6 — 7  (вес = 6)
  7 — 10  (вес = 4)
  10 — 8  (вес = 3)
  8 — 9  (вес = 6)

Суммарный вес остовного дерева: 31
```

Приложение Б

Полный код программы.

```
"""
ПРОГРАММА ПОСТРОЕНИЯ МИНИМАЛЬНОГО ОСТОВНОГО ДЕРЕВА (АЛГОРИТМ ПРИМА)

На вход подаётся матрица 10×10, содержащая целые неотрицательные числа – веса рёбер.
0 означает отсутствие ребра (или петлю). Граф неориентированный, матрица симметрична.
Программа выводит исходную матрицу, список рёбер, вошедших в остовное дерево,
и суммарный вес дерева.
"""

def read_matrix_from_file(filename):
    """Чтение матрицы 10×10 из текстового файла.
    Ожидается 10 строк, в каждой 10 целых чисел, разделённых пробелами.
    """
    try:
        # Открываем файл и построчно собираем матрицу смежности.
        with open(filename, 'r', encoding='utf-8') as f:
            matrix = []
            for line_num, line in enumerate(f, 1):
                line = line.strip()
                if not line:
                    continue      # пропускаем пустые строки
                parts = line.split()
                if len(parts) != 10:
                    raise ValueError(f"Строка {line_num} содержит {len(parts)} чисел,
а должно быть 10")

                # Преобразуем все элементы строки в целые числа.
                row = [int(x) for x in parts]

                # Проверка на неотрицательность
                if any(x < 0 for x in row):
                    raise ValueError(f"Строка {line_num} содержит отрицательные
числа")
                matrix.append(row)

            # После чтения убеждаемся, что действительно получили 10 строк.
            if len(matrix) != 10:
                raise ValueError(f"Файл должен содержать 10 строк, найдено
{len(matrix)}")

            # Проверка симметричности (необязательно, но полезно)
            # Для неориентированного графа матрица должна быть симметрична:
            # weight[i][j] == weight[j][i].
            for i in range(10):
                for j in range(10):
                    if matrix[i][j] != matrix[j][i]:
                        print("Предупреждение: матрица не симметрична, программа будет
использовать верхний треугольник")
```

```
                        break
                else:
                    continue
                break
            return matrix
    except Exception as e:
        print(f"Ошибка при чтении файла: {e}")
        return None

def print_matrix(matrix, title="Матрица расстояний"):
    print(f"\n{title}:")
    print("-" * 40)

    # Выводим матрицу в виде таблицы с выравниванием по ширине.
    for row in matrix:
        print(" ".join(f"{x:3}" for x in row))
    print("-" * 40)

def prim(matrix, start=0):
    """
    Алгоритм Прима для нахождения минимального остовного дерева.
    Возвращает (список рёбер, общий вес).
    Каждое ребро представляется кортежем (вершина1, вершина2, вес).
    Вершины нумеруются от 0 до 9.
    """
    n = len(matrix)

    # visited[v] == True означает, что вершина v уже включена в остов.
    visited = [False] * n

    # Начинаем построение дерева с вершины start.
    visited[start] = True

    # Здесь будем накапливать выбранные рёбра остова.
    mst_edges = []

    # Общий вес минимального остовного дерева.
    total_weight = 0

    # В связном графе остов для n вершин всегда содержит n - 1 ребро.
    for _ in range(n - 1):
        # На каждой итерации ищем самое дешёвое ребро,
        # которое соединяет уже посещённую вершину с непосещённой.
        min_weight = float('inf')
        from_vertex = -1
        to_vertex = -1

        # Ищем минимальное ребро из уже включённых вершин
        # в вершину, которая ещё не входит в остов.
        for u in range(n):
            # Рассматриваем только вершины, которые уже попали в дерево.
            if not visited[u]:
```

```
                continue
            for v in range(n):
                w = matrix[u][v]

                # Пропускаем:
                # 1) вершины, которые уже находятся в остове;
                # 2) отсутствующие рёбра (в матрице это 0).
                if visited[v] or w == 0:
                    continue

                # Если нашли ребро легче текущего кандидата,
                # запоминаем его как лучшее на этой итерации.
                if w < min_weight:
                    min_weight = w
                    from_vertex = u
                    to_vertex = v

        # Если подходящее ребро не найдено, значит из текущей компоненты
        # нельзя дойти до остальных вершин: граф несвязный.
        if to_vertex == -1:
            print("Внимание: граф несвязный, построено только частичное дерево.")
            break

        # Добавляем найденное минимальное ребро в остов,
        # помечаем новую вершину как посещённую
        # и увеличиваем суммарный вес дерева.
        visited[to_vertex] = True
        mst_edges.append((from_vertex, to_vertex, min_weight))
        total_weight += min_weight

    return mst_edges, total_weight

def main():
    print("=" * 60)
    print("МИНИМАЛЬНОЕ ОСТОВНОЕ ДЕРЕВО (АЛГОРИТМ ПРИМА)")
    print("=" * 60)

    # Выбор источника матрицы
    while True:
        print("\nВыберите способ ввода матрицы:")
        print("1. Прочитать из файла")
        print("2. Использовать встроенный пример")
        choice = input("Ваш выбор (1 или 2): ").strip()

        # Вариант 1: читаем матрицу из пользовательского файла.
        if choice == '1':
            filename = input("Введите имя файла: ").strip()
            matrix = read_matrix_from_file(filename)
            if matrix is not None:
                break
            else:
                print("Попробуйте снова.")
```

```
        # Вариант 2: берём готовую тестовую матрицу из программы.
        elif choice == '2':
            # Пример матрицы 10×10
            matrix = [
                [0, 2, 0, 6, 0, 0, 0, 0, 0, 0],
                [2, 0, 3, 8, 5, 0, 0, 0, 0, 0],
                [0, 3, 0, 0, 7, 0, 0, 0, 0, 0],
                [6, 8, 0, 0, 9, 0, 0, 0, 0, 0],
                [0, 5, 7, 9, 0, 1, 0, 0, 0, 0],
                [0, 0, 0, 0, 1, 0, 4, 0, 0, 0],
                [0, 0, 0, 0, 0, 4, 0, 2, 7, 0],
                [0, 0, 0, 0, 0, 0, 2, 0, 6, 3],
                [0, 0, 0, 0, 0, 0, 7, 6, 0, 5],
                [0, 0, 0, 0, 0, 0, 0, 3, 5, 0]
            ]
            print("Используется встроенный пример.")
            break
        else:
            print("Неверный выбор. Введите 1 или 2.")

    # Сначала показываем пользователю исходную матрицу графа.
    print_matrix(matrix)

    # Затем строим минимальное остовное дерево алгоритмом Прима.
    edges, total_weight = prim(matrix)

    # Выводим список рёбер, которые вошли в дерево, и их общий вес.
    print("\nРёбра, входящие в минимальное остовное дерево:")
    if edges:
        for u, v, w in edges:
            # Для удобства вывода нумеруем вершины с 1, а не с 0.
            print(f"  {u+1} — {v+1}  (вес = {w})")
        print(f"\nСуммарный вес остовного дерева: {total_weight}")
    else:
        print("Остовное дерево не найдено (возможно, граф несвязный).")

    print("\n" + "=" * 60)
    print("готово")
    print("=" * 60)

if __name__ == "__main__":
    # Запускаем программу только если файл выполняется напрямую.
    main()
```